LavAutos oPreto

Sistema de gestión de lavadero de vehículos
Informe de entrega final

Materia: Taller de Desarrollo de Aplicaciones — IUA
Docente: Ing. Gustavo Marcelo Macarone
Ciclo lectivo: 2026
Equipo: 5
Integrante: Urrets Zavalia Fernando Jose
Comisión: A
Fecha de entrega: 06/11/2026
Versión: Entrega Final

Indice:

1. Identificación
2. Problema y alcance
3. Requerimientos
4. Análisis
5. Diseño
6. Implementación
7. Pruebas y trazabilidad
8. Cierre, aportes y fuentes

Anexo: lista de control de la entrega.

1. Identificación

Campo	Valor
Nombre del proyecto	LavAutos oPreto — Sistema de gestión de lavadero de vehículos

Equipo	5
Integrantes	Urrets Zavalia Fernando Jose
Materia	Taller de Desarrollo de Aplicaciones — IUA
Docente	Ing. Gustavo Marcelo Macarone
Comisión	A
Ciclo lectivo	2026
Fecha de entrega	06/11/2026
Repositorio / commit	No incluido

2. Problema y alcance

2.1 Problema

Un lavadero de vehículos necesita registrar a sus clientes, los vehículos de cada cliente y los lavados realizados, y conocer en todo momento en qué estado está cada lavado. Sin un sistema, esa información se dispersa en planillas o papeles, se pierden los vínculos entre cliente, vehículo, lavado y no hay control sobre qué vehículos están en proceso ni sobre la evolución de los trabajos.

2.2 Usuarios y objetivo

El usuario es el administrador o empleado del lavadero, que inicia sesión con una cuenta predeterminada. El objetivo es centralizar la gestión de clientes, vehículos y lavados, con validaciones, estados controlados y consultas de seguimiento.

2.3 Funcionalidades incluidas

- Inicio y cierre de sesión con una cuenta administradora predeterminada.
- Gestión de clientes: alta, búsqueda, consulta, edición y baja (física o lógica).
- Gestión de vehículos: alta, búsqueda, consulta, edición y baja, con catálogos en cascada.
- Gestión de lavados: registro, listado por estado, avance de estado, edición y detalle.
- Dashboard de consultas: lavados entregados por rango de fechas y clientes que no vuelven desde una fecha.
- Bloqueo de edición y baja de clientes o vehículos con lavados activos.
- Script de base de datos con esquema y datos iniciales y seed de catálogos al iniciar.

2.4 Fuera de alcance

- Gestión de usuarios y auditoría funcional de operaciones.
- Stock y pagos: el **README** los menciona como idea, pero no están implementados; se declaran fuera de alcance.
- Paginación de listados, exportaciones y reportes gráficos.
- Despliegue en Internet y aplicación móvil.

3. Requerimientos

3.1 Requisitos funcionales

ID	Función	Criterio de aceptación
RF-01	Iniciar sesión	Con credenciales válidas se crea la sesión y se responde 200; con credenciales inválidas se responde 401 con mensaje genérico y no se crea sesión.
RF-02	Cerrar sesión	Al confirmar el cierre se destruye la sesión del servidor y se redirige al login; sin sesión, los endpoints protegidos responden 401.
RF-03	Proteger endpoints y páginas	Toda ruta `/api` salvo `/api/auth/*` exige sesión; una página sin sesión redirige al login.
RF-04	Cuenta administradora predeterminada	Al iniciar el backend, si no existe la cuenta `admin` se crea con contraseña hasheada; si existe, no se duplica.
RF-05	Listar y buscar clientes	Solo se listan clientes activos; la búsqueda por nombre, apellido o DNI devuelve coincidencias y una lista vacía si no hay.
RF-06	Registrar cliente con vehículo	Con datos válidos se crean persona, cliente y vehículo; faltantes responden 400; un DNI duplicado se rechaza.
RF-07	Consultar cliente y sus vehículos	La página muestra los datos del cliente y la lista de sus vehículos; un cliente inexistente responde 404.
RF-08	Editar cliente	Se actualizan los datos personales y el estado; un DNI duplicado se rechaza; un cliente inexistente responde 404.
RF-09	Eliminar cliente	Sin lavados se elimina físicamente; con lavados no activos se aplica baja lógica (`estadoCliente = 0` y vehículos con `estadoVehi = 0`).
RF-10	Bloquear cliente con lavado activo	Si algún vehículo del cliente tiene un lavado en `En Espera`, `Lavando` o `Terminado`, `PUT` y `DELETE` responden 409 sin modificar datos.
RF-11	Dashboard: clientes que no vuelven	Con una fecha elegida se listan los clientes activos sin lavados posteriores a esa fecha.
RF-12	Listar y buscar vehículos	Se listan vehículos activos y se busca por modelo o patente, con estado sin resultados.
RF-13	Consultar vehículo y su cliente	La página muestra solo el vehículo encontrado con los datos de su cliente; un id inexistente responde 404.
RF-14	Registrar vehículo	Con datos válidos se crea el vehículo; faltantes responden 400; una patente duplicada se rechaza.
RF-15	Editar vehículo	Se actualizan patente, estado, cliente y modelo; una patente duplicada se rechaza.
RF-16	Eliminar vehículo	Sin lavados se elimina físicamente; con lavados no activos se aplica baja lógica (`estadoVehi = 0`).
RF-17	Bloquear vehículo con lavado activo	Si el vehículo tiene un lavado en `En Espera`, `Lavando` o `Terminado`, `PUT` y `DELETE` responden 409 sin modificarlo.
RF-18	Catálogos de vehículo en cascada	Al elegir categoría se cargan sus marcas y al elegir marca sus modelos; los formularios exigen la selección completa.

RF-19	Registrar lavado	Con vehículo, categoría, estado, al menos una utilería y login de sesión se crea el lavado y se responde 201.
RF-20	Listar lavados por secciones	La página agrupa `En Espera`, `Lavando` y `Terminado` en secciones colapsables y no muestra `Entregado`.
RF-21	Avanzar estado del lavado	`Lavar` pasa a `Lavando`, `Completado` a `Terminado` y `Entregar` a `Entregado`, con popup y recarga de la lista.
RF-22	Editar lavado activo	Se precargan categoría, estado, detalles y utilerías; `Registrar` actualiza y `Cancelar` no guarda.
RF-23	Ver detalle del lavado	`+info` muestra patente, categoría, modelo, marca y datos del cliente, y `Aceptar` cierra la ventana.
RF-24	Actualizar utilerías	Tildar o destildar una utilería actualiza `estadoUtileria` en el catálogo mediante `PUT /api/utilerias/:id`.
RF-25	Dashboard: lavados entregados por rango	Con `Desde` y `Hasta` se listan solo lavados `Entregado` con `fechaHoraLavadoTerminado` dentro del rango.
RF-26	Bloquear LAVAR con lavado activo	Si el vehículo tiene un lavado activo, `LAVAR` muestra un aviso y no navega al alta de lavado.
RF-27	Marcas de tiempo del lavado	El backend genera `fechaHoraLavado` y `fechaHoraLavadoTerminado`; al cambiar el estado actualiza la segunda y conserva la primera.
RF-28	Impedir eliminar lavados	`DELETE /api/lavados/:id` responde 405 y el lavado permanece almacenado.
RF-29	Home con accesos y fecha/hora	El home muestra los cuatro accesos y la fecha y hora reales de Buenos Aires.
RF-30	Navegación del home	`CLIENTES`, `VEHICULOS`, `LAVADOS` y `DASHBOARD` navegan a sus páginas.
RF-31	Script de base de datos	El script crea la base, las tablas, claves y relaciones e inserta los datos iniciales de forma idempotente.
RF-32	Seed de catálogos al iniciar	El backend garantiza utilerías, categorías y estados de lavado sin duplicar los existentes.
RF-33	Dashboard: modos y registros	Los registros inician deshabilitados en gris; el botón seleccionado queda verde y habilita su registro.

3.2 Historias de usuario

ID	Historia	RF
HU-01	Como empleado, quiero iniciar y cerrar sesión para proteger la información del lavadero.	RF-01, RF-02, RF-03, RF-04
HU-02	Como empleado, quiero buscar clientes activos para encontrarlos rápido.	RF-05
HU-03	Como empleado, quiero registrar un cliente con su vehículo para comenzar a operar.	RF-06, RF-18
HU-04	Como empleado, quiero consultar y editar los datos de un cliente y sus vehículos.	RF-07, RF-08, RF-13, RF-15

HU-05	Como empleado, quiero dar de baja clientes y vehículos sin perder el historial de lavados.	RF-09, RF-10, RF-16, RF-17
HU-06	Como empleado, quiero registrar un lavado con su categoría, utilería y estado.	RF-19, RF-24, RF-27
HU-07	Como empleado, quiero seguir los lavados por estado y avanzarlos hasta la entrega.	RF-20, RF-21, RF-23, RF-26, RF-28
HU-08	Como empleado, quiero corregir los datos de un lavado activo.	RF-22
HU-09	Como encargado, quiero consultar lavados entregados por período y clientes que no vuelven.	RF-11, RF-25, RF-33
HU-10	Como responsable técnico, quiero que la base se cree con datos iniciales y catálogos listos.	RF-31, RF-32
HU-11	Como usuario, quiero una pantalla de inicio con accesos y la fecha y hora actuales.	RF-29, RF-30

3.3 Requisitos no funcionales

ID	Requisito	Verificación
RNF-01	Las contraseñas se guardan con hash scrypt y sal aleatoria; nunca en texto plano.	`src/backend/utils/password.js`; prueba `tests/password.test.js`.
RNF-02	La sesión usa cookie `HttpOnly` y `SameSite`, con `Secure` configurable por entorno.	`src/backend/app.js`; pruebas de `tests/auth.test.js`.
RNF-03	La base mantiene claves primarias y foráneas y la integridad referencial.	Script SQL; `tests/models.test.js`.
RNF-04	Las respuestas JSON son consistentes y usan códigos HTTP adecuados.	`tests/app.test.js` y pruebas de integración.
RNF-05	Los errores se manejan de forma centralizada sin exponer trazas ni detalles internos.	`src/backend/middlewares/errorHandler.js`; `tests/app.test.js`.
RNF-06	El proyecto es reproducible: versiones declaradas, configuración de ejemplo y datos iniciales.	`package.json`, `src/backend/.env.example`, script SQL y seed.
RNF-07	Las validaciones del backend no dependen de las del frontend.	`src/backend/utils/validators.js`; pruebas de clientes y vehículos.
RNF-08	No se versionan secretos: `.env` está ignorado y el ejemplo usa valores ficticios.	Revisión de `.gitignore` y `.env.example`.

4. Análisis

4.1 Backlog y tareas

Épica	Tareas principales	Estado	Responsable
Base y autenticación	Esquema SQL, modelos Sequelize, login/logout, sesión, cuenta admin, seed de catálogos	Completa	Equipo
Clientes	CRUD, búsqueda, validaciones, páginas de alta/edición/información, baja lógica	Completa	Equipo
Vehículos	CRUD, catálogos en cascada, páginas de alta/edición/información, baja lógica	Completa	Equipo

Lavados	Registro, listado por estado, avance, edición, detalle, utilerías, marcas de tiempo	Completa	Equipo
Dashboard	Filtros por fecha, consultas de lavados entregados y clientes que no vuelven, avisos	Completa	Equipo
Validaciones de integridad	Bloqueo de edición/baja con lavados activos	Completa	Equipo
Documentación	Informe, código TXT y verificación de entrega	Completa	Equipo

4.2 Reglas de negocio

1. El cliente de un lavado se obtiene por `Lavado → Vehículo → Cliente`; no existe una FK directa de lavado a cliente.
2. Un vehículo no puede tener más de un lavado en estado activo (`En Espera`, `Lavando` o `Terminado`); el botón `LAVAR` se bloquea en ese caso.
3. Estados del lavado: `En Espera → Lavando → Terminado → Entregado`. Los entregados no se muestran en el listado.
4. No se puede editar ni dar de baja un cliente o vehículo con un lavado activo: el backend responde 409.
5. Si hay lavados (no activos), la baja es lógica y los lavados se conservan; sin lavados, la baja es física.
6. Eliminar un lavado no está permitido: el backend responde 405.
7. Las utilerías se actualizan en el catálogo y el lavado guarda la utilería tildada.
8. Las fechas del lavado las genera el backend; al cambiar el estado se actualiza `fechaHoraLavadoTerminado`.
9. Dashboard: los lavados se filtran por `fechaHoraLavadoTerminado`; los clientes que no vuelven, por `fechaHoraLavado`.

4.3 Riesgos y tratamiento

Riesgo	Impacto	Tratamiento
Base de datos no disponible	El backend no inicia y las pruebas de integración fallan	Verificación de conexión al arrancar, mensaje claro y documentación de requisitos
Secretos en el repositorio	Exposición de credenciales	`.env` ignorado, `.env.example` ficticio y verificación del TXT
Duplicados por reejecución del script	Catálogos inconsistentes	Inserciones con `IF NOT EXISTS` y seed idempotente
Zona horaria del servidor	Fechas de filtros corridas	Fechas locales y documentación del criterio
Datos reales en la entrega	Incumplimiento de la guía	Solo datos ficticios y sin `.env` en el TXT
Capturas pendientes	Menor evidencia visual	Se declara y se respalda con pruebas y código

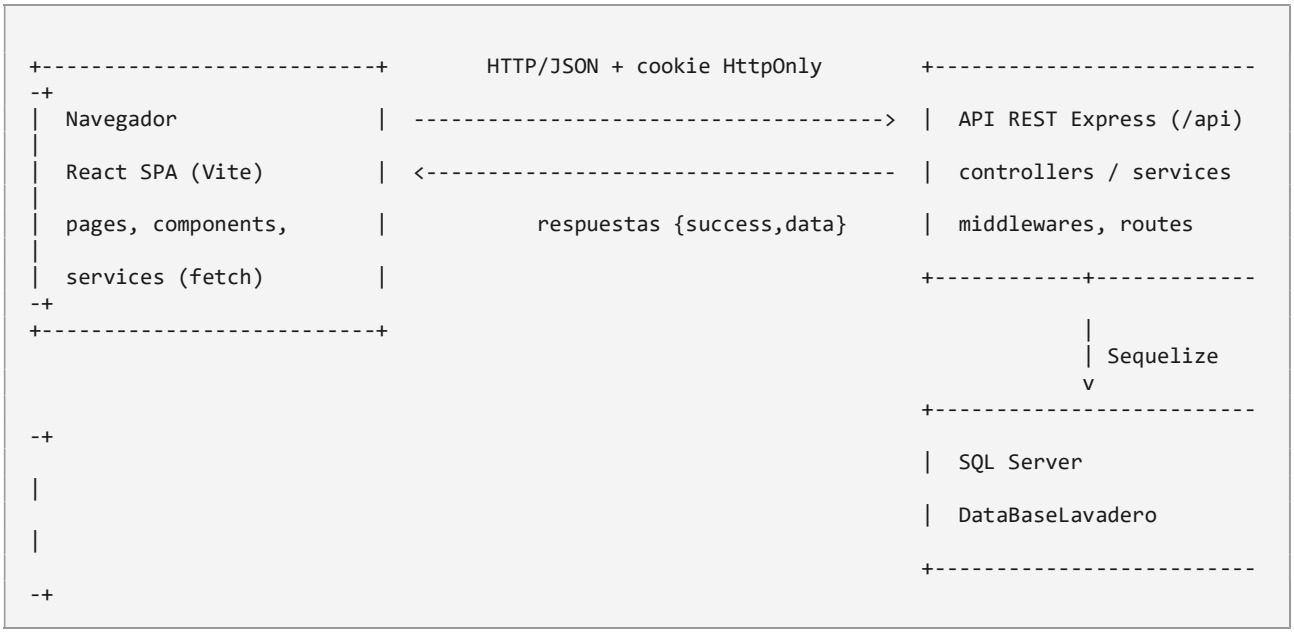
4.4 Maquetas y diferencias con la versión final

Maqueta (ruta)	Diferencia con la versión final
`src/documentacion/Prototipado/usuario/iniciar sesion.jpg`	Se mantiene el diseño; el error usa un mensaje genérico y el popup de éxito conserva el estilo.

`src/documentacion/Prototipado/home/home.jpg`	El botón `DASHBOARD` ahora navega; el resto de los accesos se mantiene.
`src/documentacion/Prototipado/cliente/clientes.jpg` y `registrar_cliente.jpg`	Se agregaron máscaras y validaciones de DNI, celular, patente y e-mail.
`src/documentacion/Prototipado/lavado/listar_lavados_activos.jpg`	Las secciones son colapsables, `+info` abre una ventana y `Editar` navega al lavado.
`src/documentacion/Prototipado/lavado/Nuevo Lavado.jpg` y `Editar Lavado.jpg`	Se guarda una sola utilería por lavado, el resto queda en el catálogo.
`src/documentacion/Prototipado/dashboard/dashboard .jpg`	Los filtros usan calendarios nativos, el estado se conserva en la URL y `+info` del cliente abre su página.

5. Diseño

5.1 Arquitectura



El navegador ejecuta una SPA de React que consume la API REST con `fetch` y cookie de sesión. La API expone rutas bajo `/api`, protege los recursos con el middleware de sesión, delega la lógica en servicios y accede a SQL Server mediante modelos Sequelize. La base concentra el esquema, las claves y los datos iniciales.

5.2 Módulos y responsabilidades

Carpeta	Responsabilidad	Archivos clave
`src/frontend/src/pages`	Pantallas completas y navegación	`LoginPage.jsx`, `HomePage.jsx`, `ClientesPage.jsx`, `ClienteInfoPage.jsx`, `NuevoClientePage.jsx`, `EditarClientePage.jsx`, `VehiculosPage.jsx`, `VehiculoInfoPage.jsx`, `NuevoVehiculoPage.jsx`, `EditarVehiculoPage.jsx`, `LavadosPage.jsx`, `NuevoLavadoPage.jsx`, `EditarLavadoPage.jsx`, `DashboardPage.jsx`
`src/frontend/src/components`	Componentes reutilizables	`AppHeader.jsx`, `BrandHeader.jsx`, `ClienteVehiculoInfo.jsx`, `ConfirmPopup.jsx`, `Popup.jsx`, `LogoutPopup.jsx`, `DateTimePanel.jsx`, `LavadoInfoPopup.jsx`
`src/frontend/src/services`	Comunicación con la API	`authService.js`, `clientService.js`, `vehicleService.js`, `lavadoService.js`

`src/frontend/src/utils`	Validaciones y formato	`validators.js`, `formatDateTime.js`
`src/backend/routes`	Definición de rutas HTTP	`index.js`, `auth.js`, `clientes.js`, `vehiculos.js`, `lavados.js`, `utilerias.js`, `estadosLavado.js`
`src/backend/controllers`	Adaptación HTTP	`authController.js`, `crudController.js`
`src/backend/services`	Reglas de negocio	`authService.js`, `clienteService.js`, `vehiculoService.js`, `lavadoService.js`, `lavadoActivoService.js`, `catalogSeedService.js`, `adminService.js`
`src/backend/models`	Persistencia Sequelize	`index.js`, `Persona.js`, `Cliente.js`, `Vehiculo.js`, `Lavado.js`, `Usuario.js`, `Login.js`
`src/backend/middlewares`	Sesión y errores	`requireAuth.js`, `errorHandler.js`, `notFound.js`
`src/databaseLavadero/script db`	Esquema y datos iniciales	`DataBaseLavadero.sql`

5.3 Modelo de datos

Tabla	Clave primaria	Claves foráneas / relaciones
Persona	codigoPersona	—
Cliente	codigoPersona	codigoPersona → Persona (TPT)
CategoriaVehiculo	codigoCategoriaVehi	—
MarcaVehiculo	codigoMarcaVehi	categoriaVehi → CategoriaVehiculo
ModeloVehiculo	codigoModeloVehi	marcaVehi → MarcaVehiculo
Vehiculo	codigoVehi	cliente → Cliente; modeloVehi → ModeloVehiculo
Usuario	codigoUsuario	—
Login	codigoLogin	usuario → Usuario
CategoriaLavado	codigoCategoriaLavado	—
EstadoLavado	codigoEstado	—
Utileria	codigoUtileria	—
Lavado	codigoLavado	vehi → Vehiculo; categoriaLavado → CategoriaLavado; login → Login; estado → EstadoLavado; utileria → Utileria

El diagrama del esquema y el modelo de datos se encuentran en `src/documentacion/Esquema Base Datos/Esquema Base Datos.png` y `src/documentacion/Modelo Datos/Modelo de Datos.png`. La relación principal es `Lavado → Vehiculo → Cliente`, sin FK directa de Lavado a Cliente.

5.4 Tecnologías y versiones

Componente	Tecnología / versión	Uso
Frontend	React 19, React DOM 19	Interfaz SPA
Build y estilos	Vite 6, Tailwind CSS 4	Desarrollo y estilos
Ruteo	React Router DOM 6.28	Navegación
Backend	Node.js 24.19, Express 4.21	API REST
Persistencia	Sequelize 6.37, tedious 16.7	Modelos y conexión SQL Server
Base de datos	Microsoft SQL Server	Almacenamiento
Pruebas backend	Jest 29, Supertest 7	Integración de la API
Pruebas frontend	Vitest 2.1, Testing Library, jsdom	Componentes y servicios
Calidad	ESLint 9, Prettier 3.4	Lint y formato

Se eligió separar frontend y backend para que la interfaz no acceda a la base y las reglas queden en la API; Sequelize facilita el mapeo y las transacciones; SQL Server responde al requisito de la cátedra; la sesión con cookie `HttpOnly` evita guardar credenciales en el navegador; Tailwind acelera la construcción de la interfaz; las pruebas automatizadas permiten la trazabilidad exigida.

6. Implementación

6.1 Árbol de carpetas

```
proyectoLavaderoAutos/
├── README.md
├── src/
│   ├── backend/
│   │   ├── app.js, server.js, package.json, .env.example
│   │   ├── config/        (database.js, index.js)
│   │   ├── controllers/   (authController.js, crudController.js)
│   │   ├── middlewares/   (requireAuth.js, errorHandler.js, notFound.js)
│   │   ├── models/        (Persona, Cliente, Vehiculo, Lavado, ...)
│   │   ├── routes/        (auth, clientes, vehiculos, lavados, utilerias, ...)
│   │   ├── services/      (auth, cliente, vehiculo, lavado, lavadoActivo, seed, admin)
│   │   ├── scripts/       (seed.js)
│   │   ├── tests/         (10 suites de Jest)
│   │   └── utils/         (AppError, asyncHandler, password, validators)
│   ├── frontend/
│   │   ├── index.html, vite.config.js, package.json
│   │   └── src/
│   │       ├── pages/       (Login, Home, Clientes, Vehiculos, Lavados, Dashboard, ...)
│   │       ├── components/  (AppHeader, Popup, ClienteVehiculoInfo, ...)
│   │       ├── services/    (auth, client, vehicle, lavado)
│   │       ├── utils/       (validators, formatDateTime)
│   │       ├── assets/      (auto.png, burbujas.png, usuario.png)
│   │       └── index.css
│   ├── databaseLavadero/script db/DataBaseLavadero.sql
│   └── documentacion/     (prototipos, diagramas, esquema, modelo)
└── openspec/              (especificaciones y cambios aprobados)
```

6.2 Tabla de funcionalidades

Funcionalidad	Estado	Observación
Login y logout	Completa	—
Cuenta administradora y seed de catálogos	Completa	—
Clientes: alta, búsqueda, consulta, edición y baja	Completa	—
Vehículos: alta, búsqueda, consulta, edición y baja	Completa	—
Lavados: registro, listado, avance, edición y detalle	Completa	—
Dashboard: consultas por fecha y avisos	Completa	—
Bloqueo por lavado activo	Completa	—
Gestión de usuarios y auditoría	No implementada	Fuera de alcance aprobado
Stock y pagos	No implementada	Mencionados en el README como idea; fuera de alcance
Paginación, exportaciones y reportes gráficos	No implementada	Fuera de alcance

6.3 Requisitos previos

- Node.js 24.19 y npm 12 (probado con esas versiones).
- Microsoft SQL Server accesible en `localhost,1433` (o la instancia configurada).
- Permisos para crear la base `DataBaseLavadero` o una base ya creada.
- Navegador actualizado; no se requiere conexión a Internet para ejecutar en local.

6.4 Instalación y ejecución

1. Crear la base y el esquema: abrir `src/databaseLavadero/script db/DataBaseLavadero.sql` en SQL Server Management Studio (o `sqlcmd -S localhost,1433 -E -C -f 65001 -i "ruta\DataBaseLavadero.sql"`) y ejecutarlo. Crea la base, las tablas, claves, relaciones y los datos iniciales de forma idempotente.
2. Configurar el backend: copiar `src/backend/.env.example` a `src/backend/.env` y completar `DB_SERVER`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_AUTH_TYPE`, `DB_DOMAIN` y `SESSION_SECRET` con valores locales ficticios. No versionar `.env`.
3. Instalar dependencias del backend: `cd src/backend` y `npm install`.
4. Opcional: cargar catálogos con `npm run seed` (el backend también los garantiza al iniciar).
5. Iniciar el backend: `npm start` (o `npm run dev`). Inicio correcto: el mensaje `Backend escuchando en el puerto 3000`.
6. Instalar dependencias del frontend: `cd src/frontend` y `npm install`.
7. Iniciar el frontend: `npm run dev`. Inicio correcto: Vite informa `Local: http://localhost:5173/`.
8. Abrir `http://localhost:5173` en el navegador. La aplicación debe mostrar el login.

El frontend usa un proxy de Vite hacia `http://localhost:3000` para `/api`.

6.5 Cuenta de demostración

Usuario **admin** y contraseña **123456** (cuenta ficticia predeterminada, creada por el backend si no existe). En un entorno real debe cambiarse por variables de entorno y una contraseña segura.

6.6 Capturas de los recorridos principales

Figura	Acción	Resultado que muestra	RF	Estado
Figura 1	Abrir la aplicación e iniciar sesión con la cuenta ficticia	Login y llegada al home	RF-01, RF-29	Terminado
Figura 2	Buscar un cliente y abrir su información	Resultados y card del cliente con sus vehículos	RF-05, RF-07	Terminado
Figura 3	Registrar un cliente con su vehículo	Formulario con catálogos en cascada y confirmación	RF-06, RF-18	Terminado
Figura 4	Editar un cliente	Datos precargados y guardado exitoso	RF-08	Terminado
Figura 5	Intentar eliminar un cliente con lavado activo	Aviso de vehículo en estado de lavado	RF-10	Terminado
Figura 6	Registrar un lavado desde un vehículo	Formulario y confirmación de registro	RF-19, RF-24	Terminado
Figura 7	Avanzar un lavado por sus estados	Secciones y popups de avance	RF-20, RF-21	Terminado
Figura 8	Abrir `+info` de un lavado	Ventana con datos del lavado y del cliente	RF-23	Terminado
Figura 9	Consultar lavados entregados por rango	Registro habilitado y resultados	RF-25, RF-33	Terminado
Figura 10	Consultar clientes que no vuelven desde una fecha	Resultados y navegación a la información del cliente	RF-11	Terminado

6.7 Partes del archivo de código

El código se entrega en tres partes para mantener cada archivo por debajo de las 5000 líneas, sin cortar ningún archivo entre partes:

Parte	Archivos	Contenido
Equipo5_Codigo_01.txt	80	README, configuración de ejemplo, dependencias, configuración de frontend y backend, script SQL y código del backend
Equipo5_Codigo_02.txt	22	Componentes y páginas del frontend
Equipo5_Codigo_03.txt	29	Resto de las páginas, pruebas, servicios, utilidades y estilos del frontend

Total: 131 archivos y aproximadamente 11.470 líneas. Se excluyeron `node_modules`, `dist`, `coverage`, `.git`, `.env`, imágenes, PDFs, DOCX, `package-lock.json`, `openspec/`, `.opencode/` y `.cursor/`.

7. Pruebas y trazabilidad

Entorno de ejecución: 2026-11-01, Node.js 24.19, SQL Server local, backend con Jest 29 y frontend con Vitest 2.1. Resultados globales: **backend 10 suites / 77 pruebas en verde** y **frontend 21 archivos / 194 pruebas en verde**. Las pruebas de integración del backend requieren SQL Server disponible.

7.1 Casos de prueba

CP	RF	Precondiciones y datos	Pasos	Esperado	Obtenido	Estado
CP-01	RF-01	Cuenta admin ficticia; credenciales inválidas	Login válido e inválido en la API y en la UI	200 con sesión; 401 con mensaje genérico	Suites `auth.test.js` y `LoginPage.test.jsx` en verde	Aprobada
CP-02	RF-02	Sesión iniciada	Cerrar sesión y consultar un endpoint protegido	Sesión destruida y 401 posterior	`auth.test.js` y `HomePage.test.jsx` en verde	Aprobada
CP-03	RF-03	Sin sesión	Llamar `/api/clientes` y abrir una página protegida	401 en la API y redirección al login	`app.test.js`, `auth.test.js`, `App.test.jsx` en verde	Aprobada
CP-04	RF-04	Base sin usuario admin	Iniciar el backend dos veces	La cuenta se crea una sola vez	`adminBootstrap.test.js` en verde	Aprobada
CP-05	RF-05	Clientes activos e inactivos	Listar, buscar por nombre/apellido/DNI y buscar sin coincidencias	Solo activos; coincidencias correctas; lista vacía	`clientes.test.js` y `ClientesPage.test.jsx` en verde	Aprobada
CP-06	RF-06	Datos válidos, faltantes y DNI duplicado	Registrar cliente con vehículo	201; 400 por faltantes; rechazo de duplicado	`clientes.test.js` y `NuevoClientePage.test.jsx` en verde	Aprobada
CP-07	RF-07	Cliente con vehículos; id inexistente	Consultar información y vehículos	Datos y vehículos; 404 si no existe	`clientes.test.js` y `ClienteInfoPage.test.jsx` en verde	Aprobada
CP-08	RF-08	Cliente existente; DNI duplicado	Editar datos y guardar	200 con cambios; rechazo de duplicado	`clientes.test.js` y `EditarClientePage.test.jsx` en verde	Aprobada
CP-09	RF-09	Cliente sin lavados y con lavados entregados	Eliminar en ambos casos	Baja física y baja lógica respectivamente	`clientes.test.js` y `ClienteInfoPage.test.jsx` en verde	Aprobada
CP-10	RF-10	Cliente con vehículo en `En Espera`	Editar y eliminar el cliente	409 sin cambios; aviso en la UI	`clientes.test.js` y `ClienteInfoPage.test.jsx` en verde	Aprobada
CP-11	RF-11	Clientes con y sin lavados posteriores	Consultar con `sinLavadosDesde`	Incluye sin lavados posteriores; excluye con lavado posterior	`clientes.test.js` y `DashboardPage.test.jsx` en verde	Aprobada
CP-12	RF-12	Vehículos activos	Listar y buscar por modelo o patente	Resultados correctos y estado sin resultados	`vehiculos.test.js` y `VehiculosPage.test.jsx` en verde	Aprobada
CP-13	RF-13	Vehículo existente e inexistente	Consultar el vehículo y su cliente	Datos correctos; 404 si no existe	`vehiculos.test.js` y `VehiculoInfoPage.test.jsx` en verde	Aprobada
CP-14	RF-14	Datos válidos, faltantes y patente duplicada	Registrar vehículo	201; 400; rechazo de duplicado	`vehiculos.test.js` y `NuevoVehiculoPage.test.jsx` en verde	Aprobada
CP-15	RF-15	Vehículo existente; patente duplicada	Editar y guardar	200 con cambios; rechazo de duplicado	`vehiculos.test.js` y `EditarVehiculoPage.test.jsx` en verde	Aprobada

CP-16	RF-16	Vehículo sin lavados y con lavados entregados	Eliminar en ambos casos	Baja física y baja lógica respectivamente	`vehiculos.test.js` y `VehiculoInfoPage.test.jsx` en verde	Aprobada
CP-17	RF-17	Vehículo con lavado en `En Espera`	Editar y eliminar el vehículo	409 sin cambios; aviso en la UI	`vehiculos.test.js` y `VehiculoInfoPage.test.jsx` en verde	Aprobada
CP-18	RF-18	Catálogos cargados	Elegir categoría y marca en los formularios	Marcas y modelos filtrados en cascada	`vehiculos.test.js` y `NuevoVehiculoPage.test.jsx` en verde	Aprobada
CP-19	RF-19	Vehículo, categoría, estado y utilería	Registrar un lavado y validar faltantes	201 con datos; aviso si falta un campo	`lavados.test.js` y `NuevoLavadoPage.test.jsx` en verde	Aprobada
CP-20	RF-20	Lavados en los tres estados y uno entregado	Abrir la página de Lavados y contraer secciones	Agrupación correcta; entregado no visible	`LavadosPage.test.jsx` en verde	Aprobada
CP-21	RF-21	Lavado en cada estado	Presionar `Lavar`, `Completado` y `Entregar`	Cambio de estado y recarga de la lista	`lavados.test.js` y `LavadosPage.test.jsx` en verde	Aprobada
CP-22	RF-22	Lavado activo	Editar datos, registrar y cancelar	PUT con los cambios; cancelar no guarda	`lavados.test.js` y `EditarLavadoPage.test.jsx` en verde	Aprobada
CP-23	RF-23	Lavado con vehículo y cliente	Abrir `+info` y aceptar	Datos completos; la ventana se cierra	`LavadosPage.test.jsx` y `DashboardPage.test.jsx` en verde	Aprobada
CP-24	RF-24	Utilería existente	Tildar y destildar una utilería	`estadoUtileria` actualizado	`utilerias.test.js` y `NuevoLavadoPage.test.jsx` en verde	Aprobada
CP-25	RF-25	Lavados entregados en distintas fechas	Consultar con `desde` y `hasta`	Solo entregados dentro del rango	`lavados.test.js` y `DashboardPage.test.jsx` en verde	Aprobada
CP-26	RF-26	Vehículo con lavado activo	Presionar `LAVAR`	Aviso y sin navegación	`ClienteInfoPage.test.jsx` en verde	Aprobada
CP-27	RF-27	Alta y cambio de estado de un lavado	Crear y actualizar el estado	Fechas generadas por el backend; terminado actualizado	`lavados.test.js` en verde	Aprobada
CP-28	RF-28	Lavado existente	Enviar `DELETE`	405 y el lavado permanece	`lavados.test.js` en verde	Aprobada
CP-29	RF-29	Sesión válida	Abrir el home	Cuatro accesos y fecha/hora de Buenos Aires	`HomePage.test.jsx` y `formatDateTime.test.js` en verde	Aprobada
CP-30	RF-30	Sesión válida	Usar los cuatro accesos del home	Cada uno navega a su página	`HomePage.test.jsx` y `App.test.jsx` en verde	Aprobada
CP-31	RF-31	Instancia SQL Server disponible	Ejecutar el script dos veces y consultar las tablas	Esquema y datos iniciales sin duplicados	Verificación manual con `sqlcmd` (2026-11-01): 3 utilerías, 3 categorías, 4 estados, 3 categorías de vehículo, 9 marcas y 9 modelos, estables tras la segunda ejecución	Aprobada

CP-32	RF-32	Base con y sin catálogos	Iniciar el backend o ejecutar el seed dos veces	Catálogos completos sin duplicados	`catalogSeed.test.js` en verde	Aprobada
CP-33	RF-33	Sesión válida	Abrir el Dashboard y alternar botones	Registros grises; botón seleccionado verde; registro habilitado	`DashboardPage.test.jsx` en verde	Aprobada

7.2 Matriz de trazabilidad

RF	Criterio de aceptación	Ruta y función/clase	CP	Estado
RF-01	Login válido e inválido	`src/backend/services/authService.js: login`; `src/frontend/src/pages/LoginPage.jsx`	CP-01	Completo
RF-02	Cierre de sesión	`src/backend/services/authService.js: logout`; `src/frontend/src/components/LogoutPopup.jsx`	CP-02	Completo
RF-03	Rutas protegidas	`src/backend/middlewares/requireAuth.js`; `src/backend/routes/index.js`	CP-03	Completo
RF-04	Admin predeterminado idempotente	`src/backend/services/adminService.js: ensureDefaultAdmin`	CP-04	Completo
RF-05	Listar y buscar clientes activos	`src/backend/services/clienteService.js: list`; `src/frontend/src/pages/ClientesPage.jsx`	CP-05	Completo
RF-06	Alta de cliente con vehículo	`src/backend/services/clienteService.js: create`; `src/frontend/src/pages/NuevoClientePage.jsx`	CP-06	Completo
RF-07	Consulta de cliente y vehículos	`src/backend/services/clienteService.js: getById`; `src/frontend/src/pages/ClienteInfoPage.jsx`	CP-07	Completo
RF-08	Edición de cliente	`src/backend/services/clienteService.js: update`; `src/frontend/src/pages/EditarClientePage.jsx`	CP-08	Completo
RF-09	Baja de cliente	`src/backend/services/clienteService.js: remove`	CP-09	Completo
RF-10	Bloqueo por lavado activo (cliente)	`src/backend/services/clienteService.js: update/remove`; `src/backend/services/lavadoActivoService.js`	CP-10	Completo
RF-11	Clientes sin lavados recientes	`src/backend/services/clienteService.js: list`; `src/frontend/src/pages/DashboardPage.jsx`	CP-11	Completo
RF-12	Listar y buscar vehículos	`src/backend/services/vehiculoService.js: list`; `src/frontend/src/pages/VehiculosPage.jsx`	CP-12	Completo
RF-13	Consulta de vehículo y cliente	`src/backend/services/vehiculoService.js: getById`; `src/frontend/src/pages/VehiculoInfoPage.jsx`	CP-13	Completo
RF-14	Alta de vehículo	`src/backend/services/vehiculoService.js: create`; `src/frontend/src/pages/NuevoVehiculoPage.jsx`	CP-14	Completo
RF-15	Edición de vehículo	`src/backend/services/vehiculoService.js: update`; `src/frontend/src/pages/EditarVehiculoPage.jsx`	CP-15	Completo

RF-16	Baja de vehículo	`src/backend/services/vehiculoService.js: remove`	CP-16	Completo
RF-17	Bloqueo por lavado activo (vehículo)	`src/backend/services/vehiculoService.js: update/remove`; `src/backend/services/lavadoActivoService.js`	CP-17	Completo
RF-18	Catálogos en cascada	`src/backend/services/categoriaVehiculoService.js`; `src/frontend/src/pages/NuevoVehiculoPage.jsx`	CP-18	Completo
RF-19	Registro de lavado	`src/backend/services/lavadoService.js: create`; `src/frontend/src/pages/NuevoLavadoPage.jsx`	CP-19	Completo
RF-20	Listado por secciones	`src/backend/services/lavadoService.js: list`; `src/frontend/src/pages/LavadosPage.jsx`	CP-20	Completo
RF-21	Avance de estado	`src/backend/services/lavadoService.js: update`; `src/frontend/src/pages/LavadosPage.jsx`	CP-21	Completo
RF-22	Edición de lavado activo	`src/backend/services/lavadoService.js: update`; `src/frontend/src/pages/EditarLavadoPage.jsx`	CP-22	Completo
RF-23	Detalle del lavado	`src/frontend/src/components/LavadoInfoPopup.jsx`	CP-23	Completo
RF-24	Actualización de utilerías	`src/backend/services/utileriaService.js: update`; `src/frontend/src/pages/NuevoLavadoPage.jsx`	CP-24	Completo
RF-25	Lavados entregados por rango	`src/backend/services/lavadoService.js: buildListOptions`; `src/frontend/src/pages/DashboardPage.jsx`	CP-25	Completo
RF-26	Bloqueo de LAVAR	`src/frontend/src/pages/ClienteInfoPage.jsx: handleLavar`	CP-26	Completo
RF-27	Marcas de tiempo del lavado	`src/backend/models/Lavado.js`	CP-27	Completo
RF-28	DELETE de lavado no permitido	`src/backend/routes/lavados.js`	CP-28	Completo
RF-29	Home con accesos y fecha/hora	`src/frontend/src/pages/HomePage.jsx`; `src/frontend/src/components/DateTimePanel.jsx`	CP-29	Completo
RF-30	Navegación del home	`src/frontend/src/pages/HomePage.jsx`; `src/frontend/src/App.jsx`	CP-30	Completo
RF-31	Script con esquema y datos iniciales	`src/databaseLavadero/script db/DataBaseLavadero.sql`	CP-31	Completo
RF-32	Seed de catálogos	`src/backend/services/catalogSeedService.js`; `src/backend/server.js`	CP-32	Completo
RF-33	Modos y registros del Dashboard	`src/frontend/src/pages/DashboardPage.jsx`	CP-33	Completo

8. Cierre, aportes y fuentes

8.1 Errores conocidos, limitaciones y pendientes

Punto	Detalle	RF afectado
Sin gestión de usuarios ni auditoría	Fuera de alcance de la primera etapa.	RF-01 a RF-04 (alcance)
Sin paginación	Los listados traen todos los registros; aceptable para el volumen actual.	RF-05, RF-12, RF-20
Stock y pagos	El README los menciona, pero no están implementados ni especificados.	—
Zona horaria	Las fechas de filtros y marcas usan la hora local del servidor.	RF-25, RF-27
Warning de documentación	La especificación «Tabla Lavado» supera el límite de longitud recomendado; no afecta la ejecución.	—
Pruebas de backend	Requieren SQL Server disponible; sin él no pueden ejecutarse.	—

8.2 Aportes del equipo

Integrante	Tareas y archivos
Urrets Z. Fernando	Creador del análisis, diseño, implementación, prueba y mantenibilidad del proyecto Lavadero Autos oPreto

El equipo revisó y validó las decisiones, la implementación y las pruebas. Las conclusiones principales son: el modelo `Lavado → Vehículo → Cliente` sostiene el dominio; los estados del lavado y el bloqueo por lavado activo evitan inconsistencias; las pruebas automatizadas permiten sostener la trazabilidad; y la documentación de la entrega queda alineada con el código real.

8.3 Fuentes y atribuciones

- React, Vite, Tailwind CSS, React Router, Express, Sequelize, tedious y SQL Server: documentación oficial de cada proyecto (consulta 2026).
- Prototipos, iconos e imágenes del proyecto: material provisto por la cátedra y el equipo, en `src/documentacion/` y `src/frontend/src/assets/`.
- No se incorporó código de terceros copiado; las dependencias se usan como paquetes declarados en `package.json`.

8.4 Uso de IA generativa

Se utilizó el asistente OpenCode (modelo deepseek-v4.1-flash) para la planificación de cambios, la generación y revisión de código, las pruebas y la redacción de esta documentación. Los aportes se verificaron con las suites automatizadas (271 pruebas en verde), la ejecución manual del script SQL y la revisión del equipo. No se compartieron datos sensibles ni credenciales reales.